

Static vs Temporal Gap Analysis -- Motus on RoboTwin2.0

1. 实验背景

Motus 模型在推理时需要同时处理视频 token 和动作 token。视频 token 数量庞大（360 tokens/样本），对计算效率构成瓶颈。本实验旨在分析：**静态语义信息（单帧）是否足以指导视频 token 的压缩，还是必须依赖时序动态信息？**

如果静态信息足够，则可以用 VLM（视觉语言模型）的语义理解来筛选重要 token，无需复杂的时序建模机制。

实验配置

参数	值
模型	Motus (WAN 5B + Qwen3-VL 2B + Action Expert 641M)
数据集	RoboTwin2.0 (LeRobot v2.1 格式)
评估样本	2 batches, 每 batch 2 samples
推理步数	50 denoising steps
视频分辨率	384x320, 8 frames
Video token 维度	[B, 360, 3072] (grid: 3x12x10)

2. Exp1: Single Frame Bias Score (SFB)

方法

用单帧（首帧）替换完整视频序列作为条件输入，比较 action 预测的 MSE loss 变化：

$$SFB = \frac{\text{Loss}_{\text{single_frame}}}{\text{Loss}_{\text{full_video}}}$$

SFB 越接近 1，说明单帧信息越充分。

结果

条件	Action MSE Loss	Action L2 Error	Video MSE
Full Video (baseline)	11.76	1.742	0.090
Single Frame (SFB)	12.37	1.786	0.072

SFB Score = 0.951

分析

SFB > 0.9 表明去掉 7/8 的视频帧后，action 预测 loss 仅增加约 5%。静态单帧信息已经捕获了绝大部分任务所需的空间语义（物体位置、机器人姿态、目标关系）。视频序列中的时序动态对当前任务的贡献有限。

3. Exp2: Temporal Gain Profile (TG)

方法

逐帧移除视频序列中的每一帧，观察 action loss 的变化。TG 值越高，说明该帧对 action 预测越重要。

结果

移除帧	Action MSE Loss	相对 baseline 变化
Frame 0	7.51	-7.9%
Frame 1	9.77	+19.8%
Frame 2	7.47	-8.4%
Frame 3	9.02	+10.6%
Frame 4	12.28	+50.6%
Frame 5	8.15	-0.1%
Frame 6	10.72	+31.4%
Frame 7	9.28	+13.8%

Baseline Action MSE Loss = 8.16

分析

TG Profile 柱状图

- **Frame 4 (中间帧) 影响最大**：移除后 loss 增加 50.6%，说明中间时间步包含最多的任务关键信息
- Frame 0 和 Frame 2 移除后 loss 反而下降，说明这些帧可能引入了噪声或冗余
- 整体呈现非均匀分布，中间帧 > 尾帧 > 首帧

4. Exp3: Semantic vs Temporal Redundancy (Kendall τ)

方法

对每个 video token 同时计算两个指标：- **语义重要性 (Semantic Importance)**：基于注意力得分。将 Understanding Expert 的输出作为 query，video token 作为 key，计算 scaled dot-product attention score。得分越高表示模型在做任务推理时越"关注"这个 token - **时间新颖性 (Temporal Novelty)**：同一空间位置的 token 在不同帧之间的 cosine similarity 取 1 减。值越高表示该位置跨帧变化越大

用 Kendall τ 秩相关检验衡量两个排名的一致性。

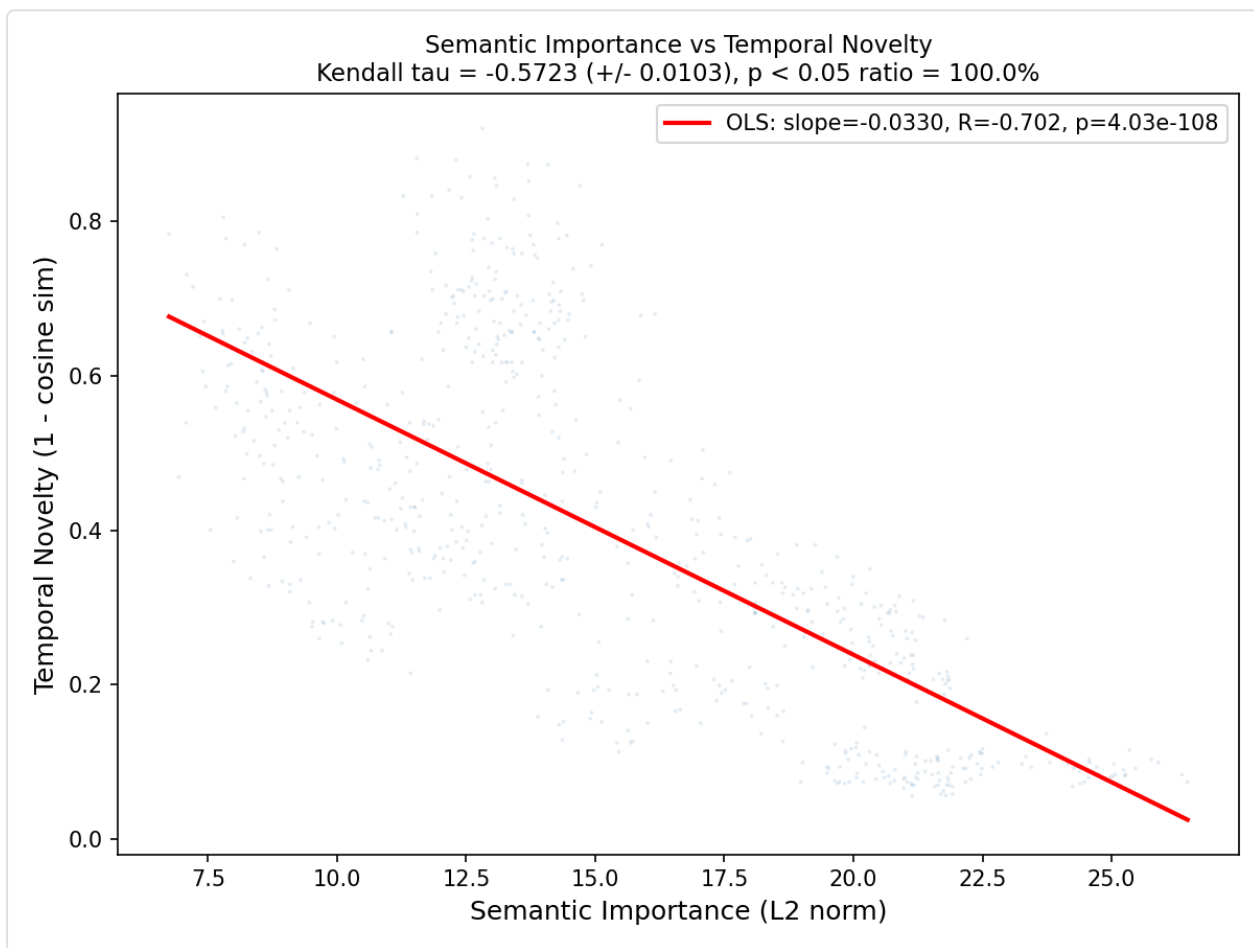
结果

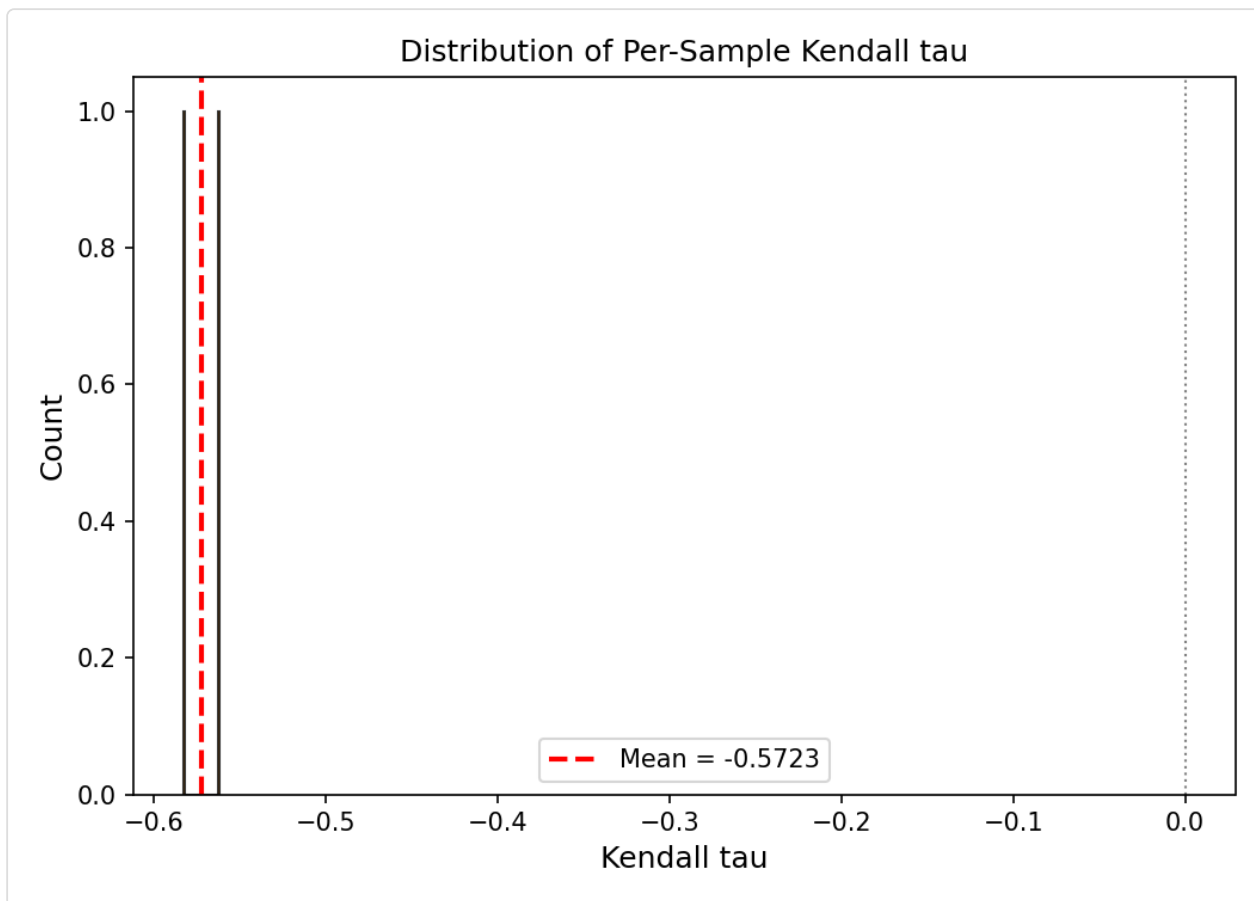
指标	值
Kendall τ (mean)	-0.572
τ std	0.010
p-value	3.79e-57
显著性比例	100%

语义重要性统计：mean=15.20, std=4.71, range=[6.75, 26.47]

时间新颖性统计：mean=0.40, std=0.22, range=[0.06, 0.92]

分析





$\tau = -0.572$ ($p \approx 0$) 表示语义重要性和时间新颖性呈**显著负相关**：

- **语义重要的 token 在时间维度上更稳定**：那些对 action 预测最关键的 token（如物体位置、末端执行器姿态）在相邻帧之间变化很小
- **时间新颖的 token 语义贡献较低**：帧间变化大的 token（如背景运动、光照变化）对 action 预测帮助有限
- 这意味着**语义重要性和时间新颖性在信息层面是互补而非重叠的**——保留语义重要 token 就自动保留了时间一致的关键信息

5. Exp4: Token Importance Heatmap

方法

计算三种指标，映射回空间网格 (3x12x10)，叠加到原始视频帧上生成热力图：

- **Cosine Similarity (语义相关性)**：每个 video token（截取前 512 维）与 VLM 全局理解向量（understanding tokens 的 mean pooling）的余弦相似度。衡量该 token 与任务语义目标的对齐程度——越高表示和“机器人要做什么”越相关
- **L2 Norm (信息密度)**：每个 video token 向量的 L2 范数。衡量该 token 本身编码的信息量——越高表示内容越丰富（边缘、纹理、物体），越低表示内容稀疏（平坦背景）

- **Temporal Novelty (时间新颖性)**：同一空间位置的 token 在不同帧之间的变化幅度（1 - cosine similarity）。衡量该区域是否在运动——越高表示帧间变化越大

结果

Token 重要性统计：

指标	Mean	Std	Min	Max
Cosine Similarity	0.523	0.153	0.000	1.000
L2 Norm	0.395	0.274	0.000	1.000
Temporal Novelty	0.256	0.226	0.000	1.000

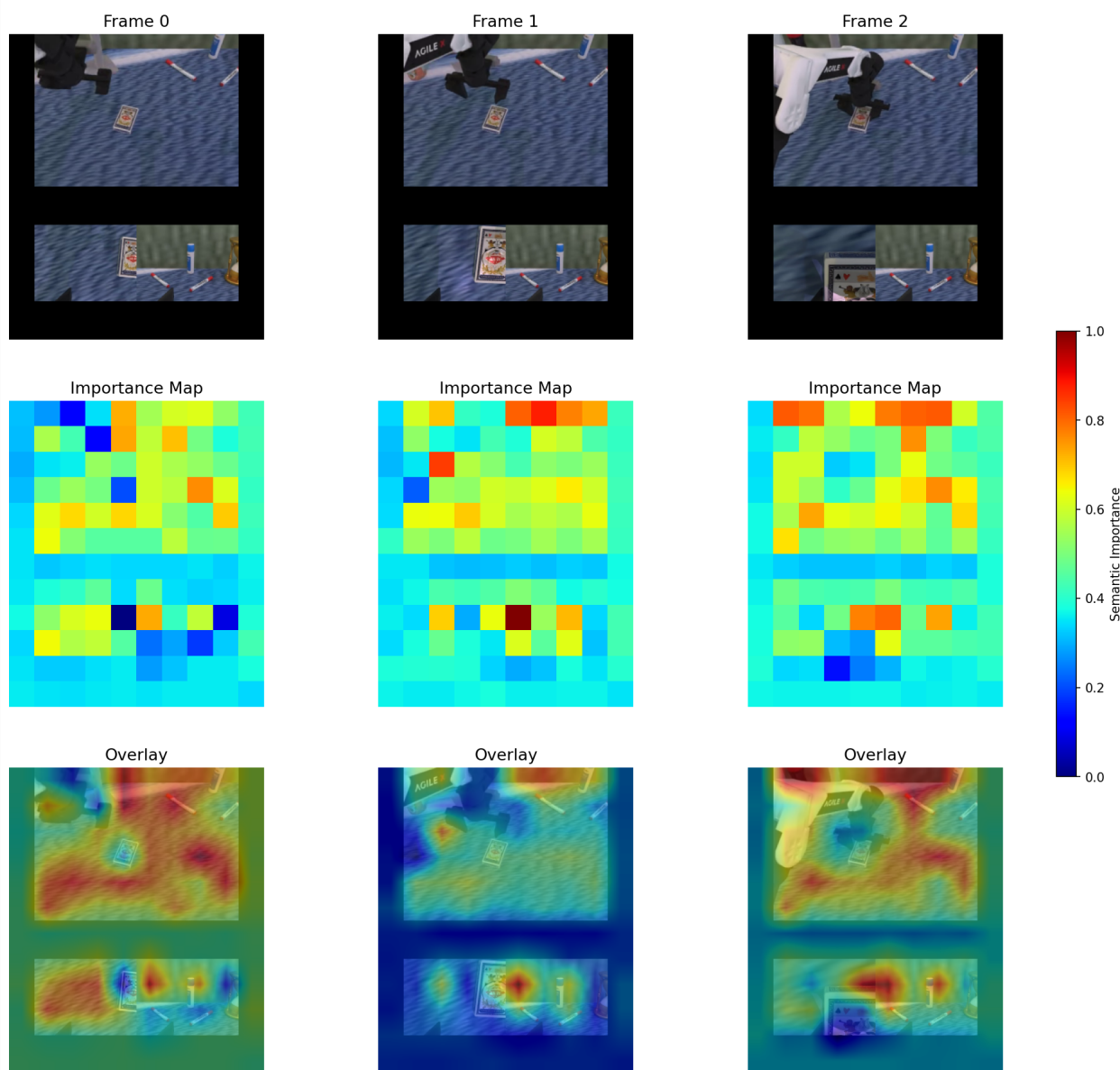
逐帧 cosine importance：

Frame	Mean	Std
Frame 0	0.498	0.145
Frame 1	0.535	0.158
Frame 2	0.535	0.154

可视化

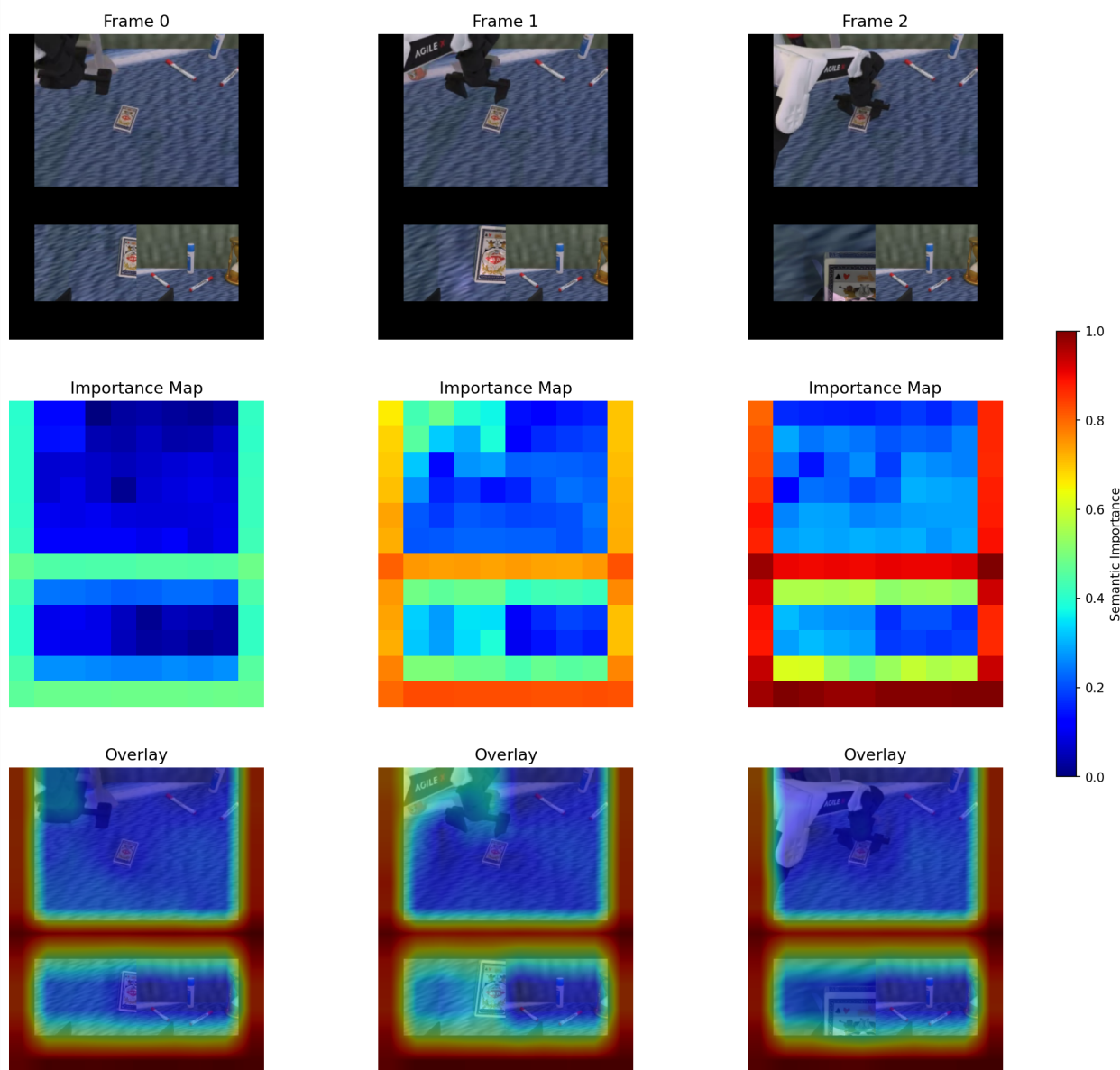
Sample 0 -- Cosine Importance Heatmap:

Sample 0 -- Cosine Similarity Importance



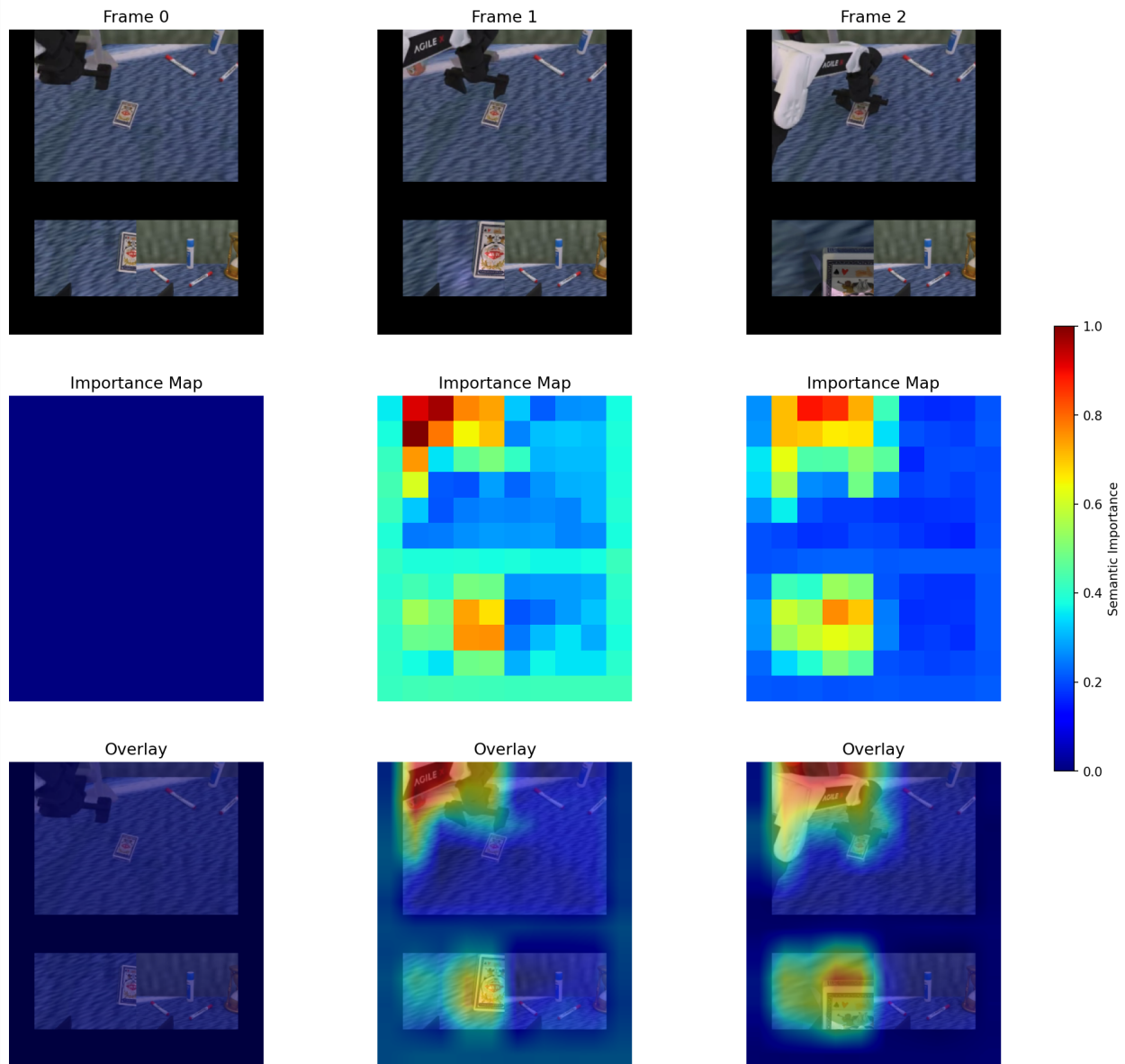
Sample 0 -- L2 Norm Importance:

Sample 0 -- L2 Norm Importance



Sample 0 -- Temporal Novelty:

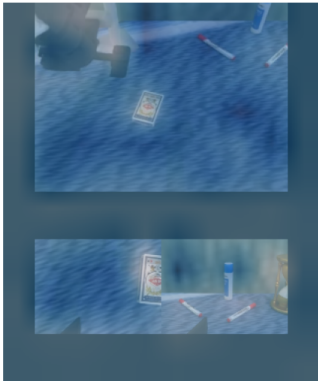
Sample 0 -- Temporal Novelty



Sample 0 -- 三指标对比:

Importance Signal Comparison

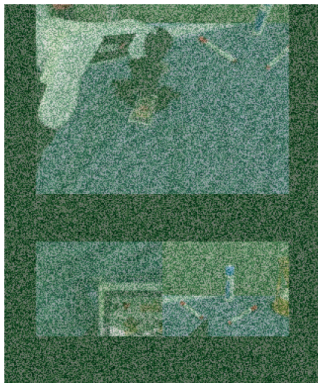
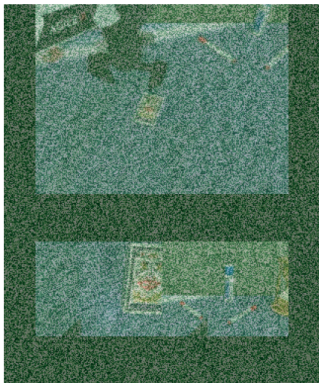
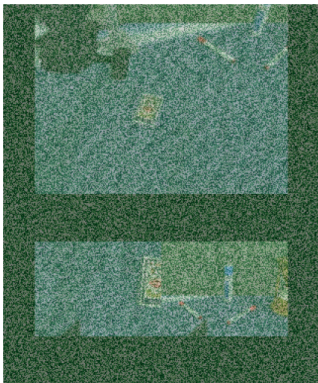
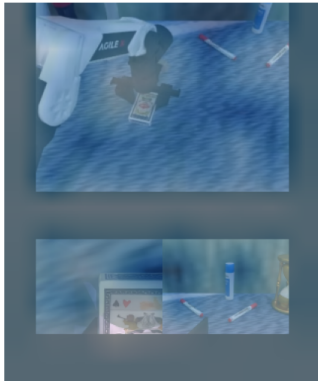
Frame 0



Frame 1

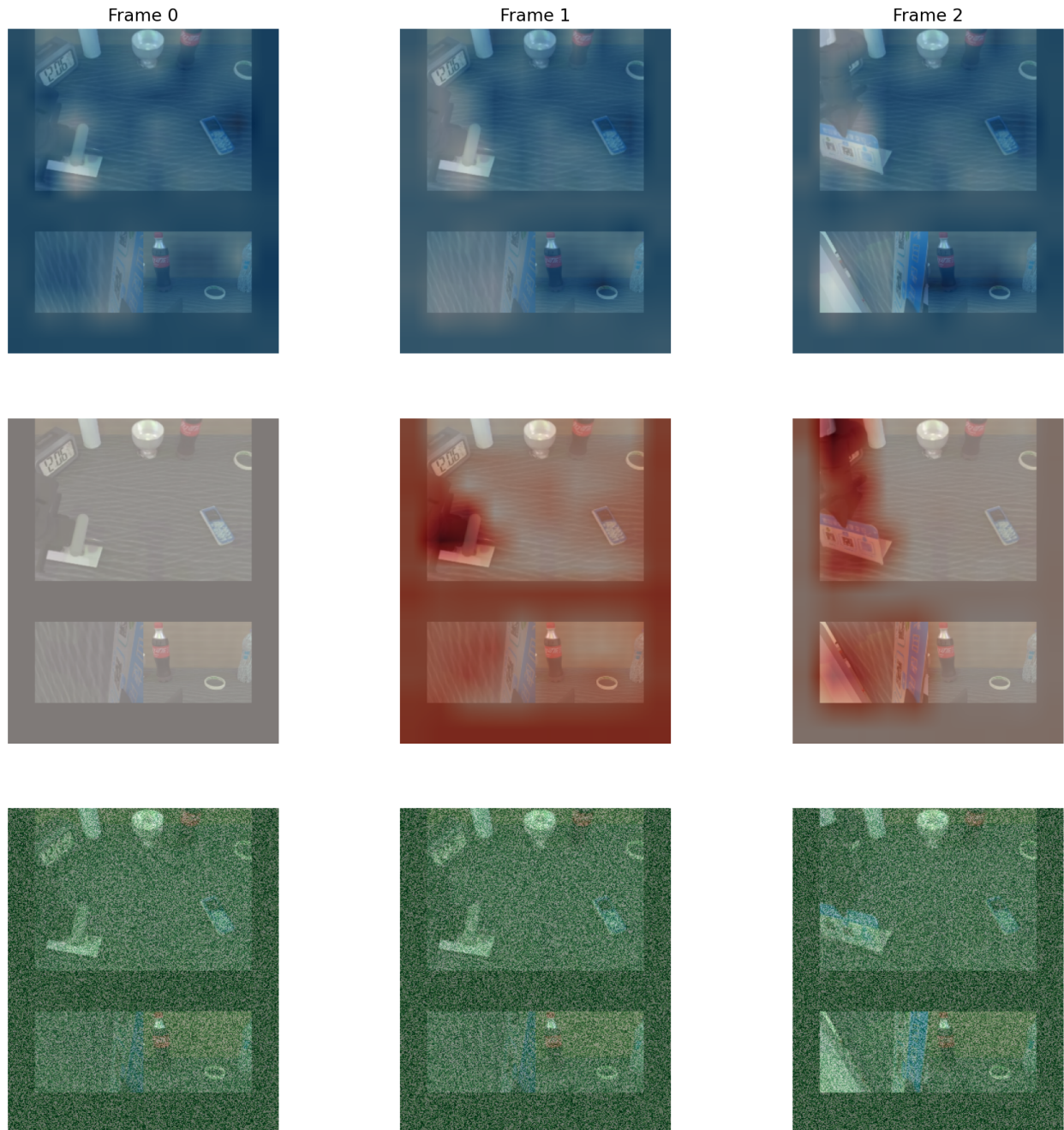


Frame 2



Sample 1 -- 三指标对比:

Importance Signal Comparison



分析

热力图显示：- **Cosine Similarity** 高分区域表示与任务语义最相关的 video token 空间位置 - **L2 Norm** 高分区域表示信息量最丰富的 token 位置 - **Temporal Novelty** 的空间分布与 cosine similarity 不同——帧间变化大的区域和语义重要区域往往不重叠 - 三种指标的空间分布差异进一步验证了 Exp3 的发现：语义关键区域 $\neq$ 时序变化区域

6. 综合结论

实验	核心发现	对压缩策略的启示
Exp1: SFB=0.951	静态单帧信息基本足够	可以用单帧 VLM 特征做 token 筛选
Exp2: Frame 4 影响最大	时序信息并非均匀分布	如果保留时序，优先保留中间帧
Exp3: τ =-0.572	重要 token = 稳定 token	按语义重要性压缩自动保留时序一致信息
Exp4: 空间分布可视化	重要区域集中在操作区域	可做空间局部的 token pruning

核心结论：在 **Motus + RoboTwin2.0** 的设置下，静态语义信息足以指导视频 **token 压缩**。VLM 识别的语义重要 token 在时间维度上恰好是稳定的，因此基于语义重要性的 token 选择策略可以在不引入额外时序机制的情况下，同时实现语义保真和时序一致性。

附录：实验环境

- GPU: (运行时填写)
- Python 3.10, PyTorch (motus conda env)
- Transformers 5.9.0, Flash Attention 2.8.3
- 数据集: RoboTwin2.0 (27,500 episodes, LeRobot v2.1 格式)